

Final Report - Biometric Synthesizer

Conrad Nelson, David Delgado, Sahil Pandit

Team 32

Spring 2026

Overview:

Most musical instruments require many months of practice before they become truly expressive. As busy students juggling stress, deadlines, and other responsibilities, we often lack the opportunity to develop these abilities, even if we are interested in music as a form of relaxation or creative expression. This creates a barrier between people and the emotional benefits that music can provide.

Our solution is a biometric synthesizer that removes this barrier by transforming the body's natural physiological signals into dynamic musical expression. Instead of relying on learned technique, the system allows users to create music through involuntary inputs such as heart rate, muscle activity, and touch. By turning internal states into sound, our device enables a more immediate and personal form of music making. One that is accessible to the user's physical and emotional state.

Description:

The system architecture of the biometric synthesizer is centered around an ATmega328P microcontroller that serves as the core processing unit. Multiple biometric sensors including a heart rate sensor, pressure sensor, muscle EMG sensor, and skin conductance sensor all collect real-time physiological data from the user. These signals are transmitted to the microcontroller, where they are processed and mapped to corresponding musical control parameters such as tempo, dynamics, pitch, and sound texture. After processing, the microcontroller converts the biometric data into properly formatted MIDI messages. These MIDI signals are then transmitted via UART to the Behringer Pro VS Mini synthesizer, which generates audio based on the received control data. This allows the user's physiological state to directly influence the resulting sound output. In addition to audio generation, the system provides real-time visual feedback through an LCD display. The display shows sensor readings and the corresponding control signals, allowing users to better understand how their biometric inputs are affecting the music.

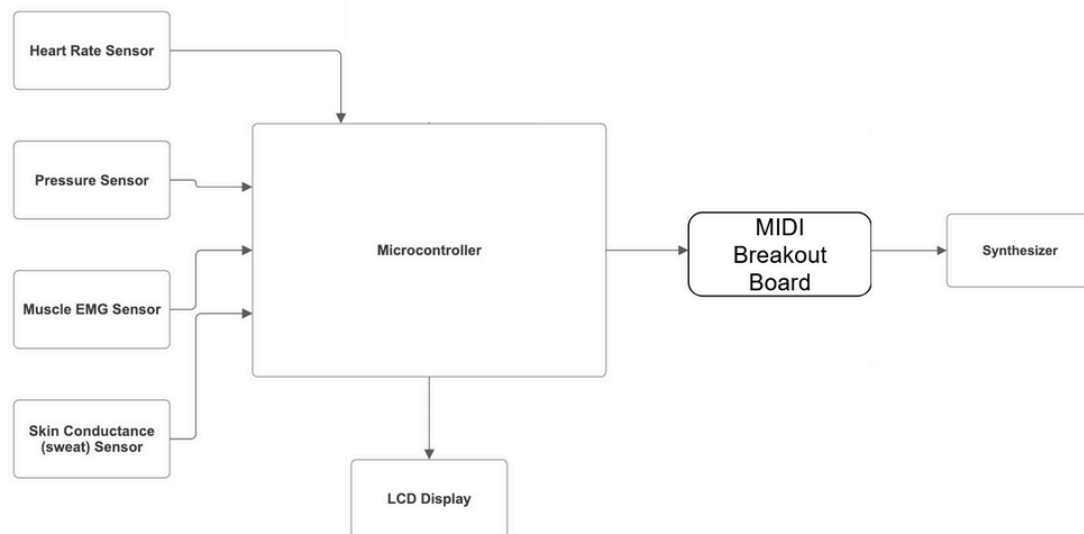


Figure 1: System Block Diagram

Power Supply Design:

Power is provided to the system through external banana plug connectors, which serve as the primary interface to a regulated +5V power source. These inputs are routed through a 3-terminal connector. A switch is included along the +5V line to allow the user to easily power the system on and off. To ensure stable operation, a $10\mu\text{F}$ decoupling capacitor is placed between the +5V line and ground. This capacitor helps smooth out voltage fluctuations and reduces noise in the power supply. An LED indicator, connected in series with a 270Ω resistor, is used to provide visual confirmation that power is being supplied to the system.

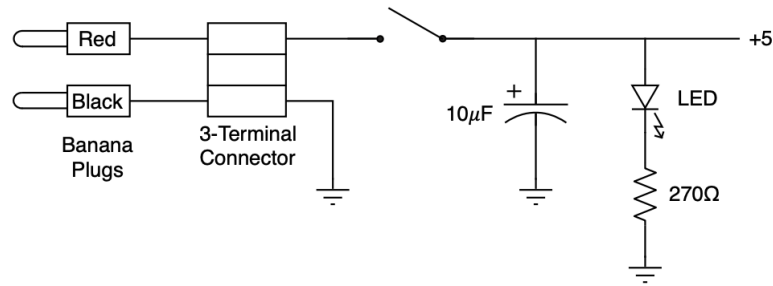


Figure 2: Power Supply Circuit Design

ATmega328P Microcontroller:

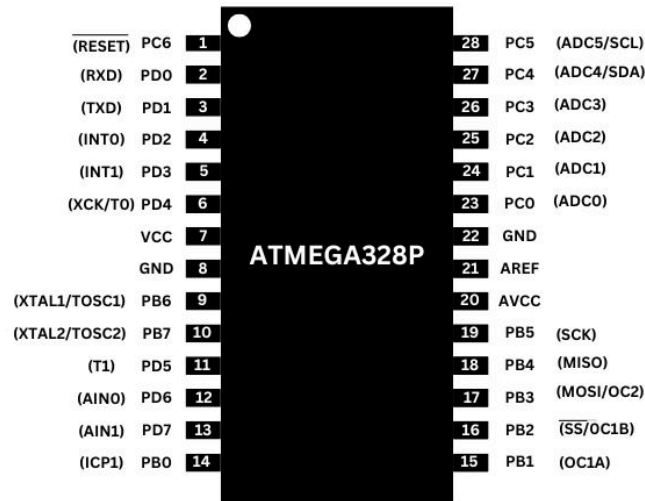


Figure 3: ATmega328P Microcontroller pins

The ATmega328P serves as the central processing unit of the system, responsible for collecting biometric sensor data and transmitting control information to the synthesizer. The microcontroller operates using an external oscillator connected to the XTAL1 pin (PB6, Pin 9) operating at a rate of **7.3728MHz**, as shown in the figure 3. The oscillator provides a stable clock

signal necessary for communication protocols such as UART and SPI. The oscillator is powered by a +5V supply and includes a decoupling capacitor (0.01 μ F) to reduce noise and ensure signal stability. To verify correct oscillator operation, an LED is connected to pin PC0 (Pin 23). This allows visual confirmation of clock-related activity during testing and debugging. The ATmega328P microcontroller is programmed using a standard 6-pin In-System Programming (ISP) connector.

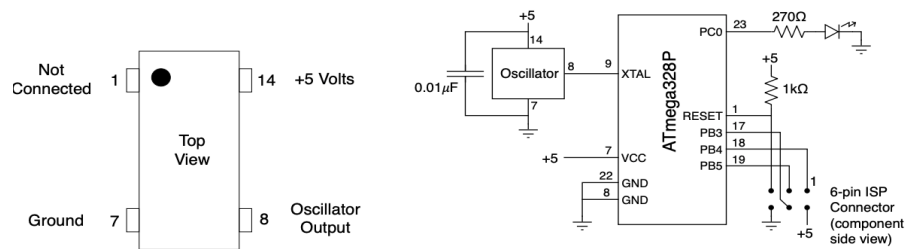


Figure 4: ATmega328P Microcontroller clock configuration

The following table is the pin mapping of the microcontroller:

Pin #	Pin Name	Function	Connected To
1	RESET	Reset	Reset Circuit
7	VCC	Power	+5V
8, 22	GND	Ground	System Ground
9	PB6 (XTAL1)	Clock Input	Oscillator
23	PC0	Digital Output	Debug LED
24	PC1	ADC	Pressure Sensor
25	PC2	ADC	EMG Sensor
26	PC3	ADC	Skin Conductance
27	PC4	ADC	Heart Rate Sensor
19	PB5	SPI CLK	LCD
17	PB3	MOSI	LCD
16	PB2	GPIO	LCD D/C

15	PB1	GPIO	LCD CS
14	PB0	GPIO	LCD Reset
3	PD1	UART TX	MIDI Output

Behringer Pro VS Mini Synthesizer:

The Behringer Pro VS Mini synthesizer is used as the primary sound generation module in the system. The synthesizer is controlled using the MIDI protocol. The ATmega328P microcontroller generates MIDI messages based on processed biometric inputs and transmits them via UART to the synthesizer. Biometric signals such as heart rate, muscle activity, pressure, and skin conductance are mapped to musical parameters including tempo, dynamics, pitch, and sound texture. An important design feature of this system is its modularity. Because the system communicates using standard MIDI messages, the Behringer Pro VS Mini can be easily replaced with any other MIDI-compatible synthesizer. This flexibility allows the system to adapt to future upgrades without requiring changes to the core hardware design.



Figure 5: Behringer Pro VS Mini Synthesizer

MIDI Communication Messages:

The Musical Instrument Digital Interface (MIDI) is a standardised serial communication interface used across digital music components, such as synthesisers and controllers, to communicate and coordinate with each other. In a typical setup, MIDI messages are sent from a designated controller/orchestrator device to one or more connected devices on a given channel and can control and modify their parameters such as pitch, volume, and timing.

In keeping with MIDI communication requirements, all MIDI messages are transmitted over UART at a baud rate of 31,250 bits per second. The rate is specifically defined by the MIDI

protocol to ensure compatibility across devices (such as the Pro Mini VS). As such we have configured our UART module on the ATmega328P to use the correct communication rate so as to reliably transmit correctly timed MIDI messages.

MIDI messages generally comprise up to three bytes. These include a status byte, which encodes both the message type and channel number, followed by one or two data bytes that contain parameter-specific information. For example, the MIDI message 0x90, 0x3C, 0x64 corresponds to a Note On message on channel 1, where 0x3C represents note 60 (middle C) and 0x64 represents the velocity value of 100.

In our project, we primarily use three types of MIDI messages: Control Change (CC), Note On/Off, and MIDI Clock messages. Each type has a specific role when it comes to translating a given biometric signal into musical output.

MIDI CC messages are used to continuously control the parameters of the synthesizer, such as volume, filter cutoff, and pitch modulation. These messages are three bytes long. The status byte is formed as $(0xB0 \mid (\text{CHANNEL_NUMBER} - 1))$, which specifies both the CC message type and the channel to be sent to. The first data byte indicates the number of the control parameter in the range 0-127, while the second data byte represents the value of the parameter (also in a 0-127 range). In our system we use CC messages to map sensor inputs of instantaneous biometric reads (such as pressure and skin conductance) to realtime adjustments of the synthesizer's parameters.

MIDI Note messages are used to trigger and release notes on the synthesizer. Like Control Change messages, they consist of three bytes. The status byte is of the form $(0x90 \mid (\text{CHANNEL_NUMBER} - 1))$ for a Note On message, and for a Note Off message $(0x80 \mid (\text{CHANNEL_NUMBER} - 1))$. The first data byte specifies the note number (0-127 range), while the second data byte represents the velocity of the note, which determines the loudness and intensity of the note. We use MIDI Note messages in conjunction with readings from our EMG sensor as a form of active, instantaneous input from the user that can be reflected in the MIDI synth.

MIDI Clock messages are a realtime synchronization message used to coordinate timing between MIDI devices. Unlike the other MIDI messages, a MIDI Clock message only consists of a single status byte 0xF8 and no data bytes. The effect of the MIDI Clock message is determined by the rate at which the messages are transmitted. In the MIDI standard, 24 Clock messages correspond to a quarter note. By controlling the frequency at which these messages are sent, the system can adjust the tempo of the synthesizer in beats per minute (BPM). We use heart rate data to dynamically control the timing of the MIDI Clock messages which in turn makes the tempo of the synthesizer passively respond to the user's physiological state.

Heart Rate Sensor:

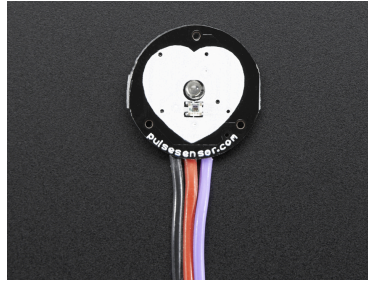


Figure 6: Adafruit 1093, Pulse Sensor Amped

In our project, a pulse sensor amp was used to measure the user's heart rate and convert it into a timed series of MIDI Clock messages that set the tempo of the connected synthesizer. The Pulse Sensor we used consists of three pins— VDD, Data, and Ground. The sensor measures the user's pulse and reports it as a voltage that can be read as a 0-1023 value by the ATmega328P ADC. The signal is processed to determine the beats per minute (BPM), which is then used to control the tempo of the synthesizer by sending MIDI Clock messages at a rate determined by the user's heart rate.

Pressure Sensor:

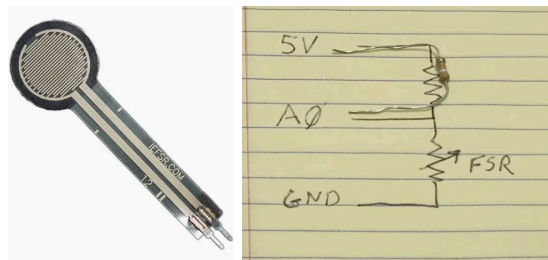


Figure 7: Adafruit 166, Force Sensitive Resistor (left); Wiring Diagram (right)

We used the Adafruit 166 Force Sensitive Resistor to allow the user to control the filter cutoff parameter of the synthesizer. The FSR was wired in series with a 10k Ohm resistor, such that the change in voltage division between the two components would reflect in the voltage at the corresponding ADC pin of the ATmega328P. Unlike the heart rate, since filter cutoff is an instantaneous control command, we can simply sample the value of the FSR and convert it to a value between 0 and 127 to be sent to the controller (MIDI CC74).

We decided it would be better to perform some basic signal processing on the FSR values by establishing lower and upper thresholds for the FSR read and to smooth step between readings so that unexpected behavior (e.g. significant noise in sensor readings or disconnections) won't lead to sporadic behavior and control of the synthesizer.

EMG Muscle Sensor:

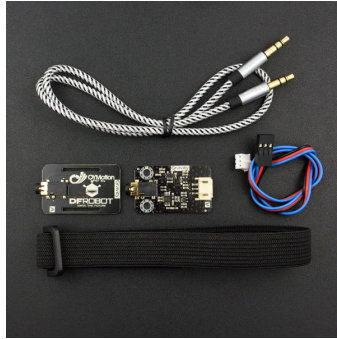


Figure 8: SEN0240, EMG Muscle Sensor

We used the SEN0240 EMG Muscle Sensor as a way for the user to directly trigger a note on the synthesizer based on flexion (e.g. of the forearm). Since the EMG sensor is responsible for Note On and Note Off messages, we opted to use a fairly simple approach to reading and using the values of the respective ADC pin. We used a simple threshold on a read along with a flag to determine whether or not a user has flexed (i.e. above threshold) and whether to send the Note On/Off (e.g. a bool for if Note On/Off has been sent for this threshold region). We debated using a more fine grained implementation of the EMG sensor that would allow us to send a note with varying velocity based on the intensity of the user's flexion (e.g. based on the rate of change of the ADC pin voltage), however when testing this we decided to revert to the simple binary approach. The main reasons for this were that there was enough noise in the readings that the produced note would not smoothly scale its velocity and that if we opted to use more coarse grained velocity bands the noise still resulted in choppy and inconsistent notes being triggered.

Skin Conductance Sensor: Seed Tech Grove GSR



Figure 8: Seed Tech Grove GSR

We used the Seed Tech Grove GSR as a skin conductance sensor to control the reverb parameter of the synthesizer. We thought it would be interesting to use skin conductance as a biometric input for our synthesizer as we felt it would make sense to have a more passive, ambient input in our system. Since the EMG and FSR sensors are on the active, direct end of the input spectrum (in terms of control/feedback) and the Pulse Sensor lies somewhere in the middle

(heart beat is passive but easily influenced), we thought it would be good to have an input on the other end of the spectrum.

Like the other inputs, the GSR sensor value is read from an ADC pin on the ATmega328P and has some basic processing applied before we convert it to a MIDI value to be sent. Similar to the FSR, the GSR is not continuous or timing dependent in the way that something like the Pulse Sensor is and so we can simply map and send the corresponding MIDI CC message (CC91).

Real Time Visual Feedback:

To enhance user interaction and understanding of the system, real-time visual feedback is provided through an LCD display. The display presents live data from the biometric sensors, allowing users to observe how their physiological signals are being captured and processed by the system. Key parameters such as heart rate, muscle activity, pressure levels, and skin conductance are visualized in real time. In addition to raw sensor values, the display can also show the corresponding control signals or mappings being sent to the synthesizer (e.g., tempo, modulation level, or intensity).

The visual feedback serves several important purposes:

- User Awareness: Helps users understand how their body is influencing the sound
- Debugging: Assists in verifying correct sensor operation and signal integrity
- Engagement: Makes the system more interactive and intuitive

Putting it All Together (The Code):

This section will briefly go over the organization of all the code used in the project and then particularly focus on and elaboration upon the main control loop and how the previous design decisions have been implemented.

The codebase is organized as multiple C source and header files, each of which is responsible for a specific subcomponent of the biometric synthesizer; the relevant files are:

- `adc_read.c/h`
- `font.c/h`
- `main.c/h`
- `midi_output.c/h`
- `process_input.c/h`
- `spi_display.c/h`

`adc_read.c/h`

Our `adc_read.c/h` is a simple utility header that handles the initialization of the ATmega328P's Analogue to Digital Converter (ADC) module and implements an `adc read` function. The `uint16_t get_adc(uint8_t channel)` function reads the value of the ADC at the specified channel/pin and as the means by which we get all our input sensor readings.

font.c/h

We implemented a simple and stripped back font header used to define the corresponding pixel values for rendering text on our SPI Display. While an entire library existed for the display we used, it is written in C++ and also contains a lot of functionality we didn't need and so decided to write a barebones version of an interface and font.

The font header simply defines the pixel arrangement (as a bitmap) for the set of characters we used with a fixed font size of 5x7.

E.g. `static const uint8_t glyph_A[5] = {0x7E, 0x11, 0x11, 0x11, 0x7E};` and `const uint8_t* get_glyph(char c);`

`get_glyph()` will return a pointer to the statically allocated 5x7 bitmap of the inputted character and using this simple approach provides sufficient functionality for displaying the sensor values and MIDI information with minimal memory use and overhead.

spi_display.c/h

The `spi_display` module handles all communication with the LCD display over the SPI protocol—it handles initializing the display, sending data and commands, and rendering our signal information as text on the screen.

```
9  typedef enum {
10      Raw = 0,
11      Biometrics = 1,
12      MIDI = 2
13  } ScreenState;
14
15  typedef struct {
16      // raw inputs
17      uint16_t pulse_raw;
18      uint16_t pressure_raw;
19      uint16_t emg_raw;
20      uint16_t gsr_raw;
21
22      // processed biometrics
23      uint16_t pulse_proc;
24      uint16_t pressure_proc;
25      uint16_t emg_proc;
26      uint16_t gsr_proc;
27
28      // midi messages
29      uint8_t pulse_midi;
30      uint8_t pressure_midi;
31      uint8_t emg_midi;
32      uint8_t gsr_midi;
33  } ScreenInfo;
```

Figure 9: `ScreenState` and `ScreenInfo` Structs Used to Encapsulate Screen Data

The main functionality of the module includes:

- Configuring and initializing communication over SPI (clock, data order, etc.)
- Sending the display specific (ST7789) initialization instructions (in *spi_init()*)
- Write character/pixel data to the buffer for display using the font module

The key methods of the module are:

- *void spi_init(void);*
- *void spi_display_init(void);*
- *void spi_display_draw_string(uint16_t x, uint16_t y, const char* s, uint16_t foreground, uint16_t background);*
- *void spi_display_draw_SCREEN(const ScreenInfo* s_info);* // SCREEN can be raw, biometrics, or midi depending on the information to be displayed

The display operates at a fairly low level in that we, for example, directly convert character data to pixels. In keeping with the reasoning behind implementing our own barebones font library, we decided to implement our own simple SPI display driver. This is because it allows for easier and lightweight integration with the rest of the system. Additionally there are the same issues of the difference in language of the existing library and excess features and overhead.

midi_output.c/h

Our midi_output module is responsible for formatting and transmitting MIDI messages over UART. We've implemented the module in a way that it abstracts the details of MIDI message construction into a MIDI_Message struct so that later references to MIDI messages can be done in terms of parameters instead of raw byte sequences.

```
typedef struct {  
    uint8_t length; // if length > 1, then not MIDI CLOCK  
    uint8_t data[3]; // 0: type, 1: data1, 2: data2  
} MIDI_Message;
```

Figure 10: MIDI_Message Struct Definition

We also defined and implemented the following methods to handle the packaging and transmission of MIDI messages over UART from the ATmega328p to the MIDI breakout board:

- *void init_UART(void);*
- *void send_UART(uint8_t);*
- *MIDI_Message get_MIDI_message(int sensor, int val);*
- *void send_MIDI(const MIDI_Message* msg);*

process_inputs.c/h

The process_inputs module is responsible for converting raw ADC values into MIDI mappable parameter values to control parameters like filter cutoff and reverb. In it we define a single function `uint8_t MIDI_map_value(uint8_t sensor, uint16_t sensor_val);` that takes an input ADC read value, process it (e.g. ensure it's at or above minimum threshold value), and then return a scaled, MIDI mapped value for use. In essence the mapping is a basic scaling of the 0-1023 input range of the ADC read to the 0-127 range of the MIDI control value.

main.c

```
138 int main(void) {
139     adc_init();
140     init_UART();
141     spi_display_init();
142     timer1_init();
143     sei();
144
145     current_screen = Raw;
146
147     while (1) {
148         uint32_t now;
149         cli();
150         now = ms_counter;
151         sei();
152
153         if (now - last_sensor_read >= SENSOR_READ_MS) {
154             last_sensor_read = now;
155             update_raw_vals();
156             update_processed_vals();
157             update_pulse(screen_info.pulse_raw, now);
158         }
159
160         if (now - last_midi_update >= MIDI_UPDATE_MS) {
161             last_midi_update = now;
162             update_midi_out();
163         }
164
165         update_midi_clk(now);
166
167         if (now - last_display_update >= DISPLAY_UPDATE_MS) {
168             last_display_update = now;
169             update_display();
170         }
171     }
172 }
```

Figure 11: Main Control Loop

Our main.c file is responsible for integrating all our previously defined modules and contains the central control loop of the project. As seen in the figure above, our main control loop consists of three subcontrol loops that are responsible for coordinating the reading of sensor inputs, sending of MIDI messages, and updating the display. Each of these run at a separate control frequency (i.e. Sensor Read and MIDI both run at 100Hz and the Display Update at 10Hz). The reason for this is because the heaviest part of our project is communicating with and updating our LCD display, but fundamentally the focus of our project is on taking biometric inputs and converting them to MIDI messages for use with a MIDI controller or synthesizer. To that end, we decided it would be fine to run the display at a lower frequency than the rest of the system since even if the displayed information is slightly out of date, the delta between the displayed and actual values is minor; moreover, the true realtime results will be heard via the synthesizer, so even if the display is out of date the realtime validity of the results aren't compromised.

At a high level, our control loop performs the following steps:

- Read the sensor values using our ADC module
- Process and map sensor values using the input processing module
- Generate the corresponding MIDI messages
- Transmit the MIDI messages via UART to our MIDI breakout board and by extension our Synth
- Update the LCD display with the current relevant values depending on what screen is selected (e.g. Raw sensor values, Biometrics, MIDI values)

```

13 #define DEFAULT_RAW_VAL 200
14 #define MIN_VALID_SENSOR_VAL 100
15 #define MAX_VALID_SENSOR_VAL 1000
16
17 #define PULSE_HIGH 600
18 #define PULSE_LOW 400
19
20 #define SENSOR_READ_MS 10
21 #define DISPLAY_UPDATE_MS 100
22 #define MIDI_UPDATE_MS 10
23
24 static ScreenState current_screen = Raw;
25 static ScreenInfo screen_info;
26
27 volatile uint32_t ms_counter = 0;
28 static uint32_t last_sensor_read = 0;
29 static uint32_t last_display_update = 0;
30 static uint32_t last_midi_update = 0;
31
32 static uint32_t last_pulse_time = 0;
33 static uint32_t midi_clk_spacing_ms = 0; // time between MIDI clock messages
34
35 static uint16_t last_pulse_val = 0;
36 static uint16_t bpm = 60;
37
38 static uint32_t last_clock_time = 0;
39
40 static void timer1_init(void) {
41     // Clear Timer on Compare Match (CTC) mode
42     TCCR1A = 0;
43     TCCR1B = (1 << WGM12);
44
45     TCCR1B |= (1 << CS11) | (1 << CS10); // Prescaler 64
46     OCR1A = 1151; // (F_CPU = 7372800 / 64 / 100) - 1
47
48     TIMSK1 = (1 << OCIE1A);
49 }
50
51 ISR(TIMER1_COMPA_vect) {
52     ms_counter += 10;
53 }

```

Figure 12: System Timing Design and Implementation

Our timing design and implementation can be seen in the figure above, we simply coordinate the control loop using the timer interrupt on the ATmega328P. We chose a OCR1A value of 1151 since we weren't able to prescale our clock in a way that resulted in a clean 1ms interrupt rate, however given that all our control loops work in multiples of 10ms anyways it is not an issue for the scope of this system. Moreover, some of the constants and bookkeeping variables can be seen defined in the above figure which are used to keep track of and govern the main control loop and logic.

```

81 static void update_pulse(uint16_t pulse_val, uint32_t now) {
82     // Rising edge
83     if (pulse_val > PULSE_HIGH && last_pulse_val <= PULSE_HIGH) {
84         uint32_t t_interval = now - last_pulse_time;
85         last_pulse_time = now;
86
87         if (t_interval > 300 && t_interval < 2000) {
88             uint16_t updated_bpm = 60000 / t_interval;
89
90             bpm = (bpm * 3 + updated_bpm) / 4; // interpolate between old and new bpm to reduce
91
92             midi_clk_spacing_ms = 60000 / (bpm * 24);
93         }
94     }
95
96     last_pulse_val = pulse_val;
97 }
98
99 static void update_midi_clk(uint32_t now) {
100     if (now - last_clock_time >= midi_clk_spacing_ms) {
101         last_clock_time = now;
102         send_UART(MIDI_CLOCK);
103     }
104 }
105
106 static void update_midi_out(void) {
107     screen_info.pressure_midi = screen_info.pressure_proc;
108     screen_info.gsr_midi = screen_info.gsr_proc;
109     screen_info.emg_midi = screen_info.emg_proc;
110
111     MIDI_Message msg;
112
113     msg = get_MIDI_message(PRESSURE, screen_info.pressure_midi);
114     send_MIDI(&msg);
115     msg = get_MIDI_message(SKIN, screen_info.gsr_midi);
116     send_MIDI(&msg);
117     msg = get_MIDI_message(EMG, screen_info.emg_midi);
118     send_MIDI(&msg);
119 }

```

Figure 13: Pulse + Tempo Control Logic

For the most part, the majority of main.c comprises hooks to the relevant functions in the other modules described earlier, however given the time dependent nature of pulse measurements and MIDI tempo control semantics, we decided to keep all the relevant processing in main.c using the established timer interrupts. Of note are the *update_pulse()* and *update_midi_clk()* functions which handle converting our processed pulse readings into a BPM value that can be used to determine the rate at which we send out our MIDI Clock messages over UART to the breakout board/synth.

Improvements:

We struggled with some damage to our microcontroller and circuitry during our work on this project due to unexpected excessive voltages. As an improvement for the future, we would likely use protective circuitry (such as fuses, overvoltage protection, current limitation, and/or voltage regulation) in order to prevent incidents like this from repeating themselves.

We also had some trouble debugging and testing our sensors. In the future, we'd like to improve this process by utilizing sensors that have LEDs on their respective PCBs. We ran into a lot of trouble working on our project when we had to figure out whether or not certain components were broken after the aforementioned circuitry damage, and this was worsened by many components not having LEDs on themselves to show they were properly receiving power. This, combined with certain components being difficult to debug (such as the muscle EMG sensor and skin conductivity sensor), greatly increased the amount of difficulty we faced working on the project. If all of our sensors had power lights on them, it would have been much easier to see whether they were either broken, or only attached/utilized improperly by us. This would have saved us a lot of time debugging, however the sensors we found for the project unfortunately did not have LEDs on their PCBs.

On a related note, another way to improve the project for the future would be to use more easily-debuggable sensors in our construction. The muscle EMG and skin conductivity sensors were difficult to work with because their activation threshold values were very hard to find, and it was also time consuming and tedious to see how much they vary with different amounts of muscle contraction and skin conductivity, especially because skin conductivity is largely an involuntary property relating to how sweaty one's skin is. This made it particularly difficult to establish and tweak the parameters and values for these sensors in order to make them useful in the scope of our project.

We also had some difficulty running code with our circuit board and wiring due to it being finicky and sensitive to movement. This was likely because of loose wiring connections, which became hard to manage and time-consuming to debug as we increased the number of components on our circuit board. A strong way to address this issue would be to order PCBs for the circuit board's schematic so that we would never have to worry about our connections coming loose. We didn't end up doing this during the semester due to time constraints as we finished our blueprint, but this would have prevented many of the problems we faced. This PCB could also be connectable to all of our other components, and greatly reduce the size and clutter of the finished theoretical product while also making its performance more reliable.

Cost Analysis:

Component	Cost (\$)
ATmega328P Microcontroller	3
Heart Pulse Sensor	25
Applied Pressure Sensor	4
Muscle EMG Sensor	50
Skin Conductivity Sensor	12
LCD Display	18
Total	102
Behringer Pro VS Mini Synthesizer	70

The synthesizer that we used cost \$70, however our project was built to be compatible with any MIDI-based synthesizer, so it is not necessary to purchase such a synth if a prospective customer already has one. Therefore, we will omit the cost of the synthesizer we used in the total in order to show the cost of the modular design itself.

We only required one of each component in order to make our project functional.

Engineering Safety/Standards:

For our project, we sourced all of our sensory components from online vendors. Because of this, we have no direct control over whether or not any safety issues occur. If any incidents occurred where the safety of the user were compromised, then our best choice would be to utilize components from another vendor. This does seem unlikely, however, as in order to sell components officially the vendors have to pass safety checks.

It is unlikely that any health incidents would occur as a result of our own wiring/component placement within our construction, as our actions relating to the hardware were essentially just connecting each of our components for proper function. For a hypothetical mass-deployment of this product, we would likely use a PCB rather than a wired-on circuit board for reliability's sake, however this would also ensure that no faulty wiring puts user safety at risk.

Our project was intended with power outlet usage rather than battery usage in mind, so there's no risk of any sort of battery fire for the user to contend with. All that the project would require is a compatible outlet cable plug-in on the circuit board so that the user can receive power without needing a DC power supply.

We used SPI and UART protocols as described earlier in the report to enable proper functionality for our project.

Signature Sheet:

	Conrad Nelson	David Delgado	Sahil Pandit
System design	33.3%	33.3%	33.3%
Hardware design	33.3%	33.3%	33.3%
Software design	5%	5%	90%
Project report (oral)	40%	20%	40%
Project report (written)	15%	15%	70%

____Conrad Nelson_____

____David Delgado_____

____Sahil Pandit_____